

TRABALHO DE COMPILADORES

IMPLEMENTAÇÃO DE UMA LINGUAGEM DE PROGRAMAÇÃO BÁSICA

Uma apresentação de analisador sintático e léxico utilizando Bison
& Flex

Fernanda Pacheco Bento¹

PONTA GROSSA
MAIO/2026

¹ fernandabento@alunos.utfpr.edu.br



SUMÁRIO

1. APRESENTAÇÃO DA LINGUAGEM	3
2. ANÁLISE LÉXICA	3
2.1. APRESENTAÇÃO DAS REGRAS	3
2.2. IMPLEMENTAÇÃO NO FLEX	7
3. TABELA DE SÍMBOLOS	7
4. ANÁLISE SINTÁTICA	8
4.1. APRESENTAÇÃO DAS REGRAS	8
4.2. IMPLEMENTAÇÃO NO BISON	11
5. CONJUNTO DE TESTES	13
6. CONSIDERAÇÕES FINAIS	14
7. USO DE IA	14



1. APRESENTAÇÃO DA LINGUAGEM

A linguagem de programação básica desenvolvida para esse trabalho abrange as funções implementadas no exemplo da calculadora avançada em uma das apostilas da disciplina. As funções envolvem operações básicas como operações aritméticas, estruturas condicionais, estruturas de repetição e declarações de funções pelo usuário. Além das funções já existentes no código da calculadora, foi adicionado como forma de cumprimento dos requisitos do trabalho os seguintes itens:

- a) Implementação das operações lógicas `AND` e `OR`
- b) Implementação da estrutura de repetição `FOR`, com a sintaxe:

```
for(init; cond; inc) lista-comandos
```

onde:

`init` é a variável de controle

`cond` é a condição para o laço

`inc` é o incremento da variável de controle

`lista-comandos` é o bloco de execução do laço

Todos os arquivos utilizados para esse trabalho estão compactados como `.zip` e submetidos na entrega do Moodle. Além disso, também há um repositório público no Github com todo o código fonte, arquivos de apoio para esse presente relatório e os diagramas requisitados na descrição do trabalho. O repositório pode ser acessado por meio do link: <https://github.com/cafeef/COMP-T1-FernandaPachecoBento>

2. ANÁLISE LÉXICA

Para a análise léxica, o arquivo de lexer da calculadora foi adaptado para adicionar os tokens de `AND`, `OR` e `FOR` para implementação futura da regra no arquivo de parser. A seguir, serão explicadas as regras mais complexas do analisador léxico e em seguida uma tabela para descrição e padrões dos outros tokens.

2.1. APRESENTAÇÃO DAS REGRAS

Para o analisador léxico, as regras foram divididas em categorias:



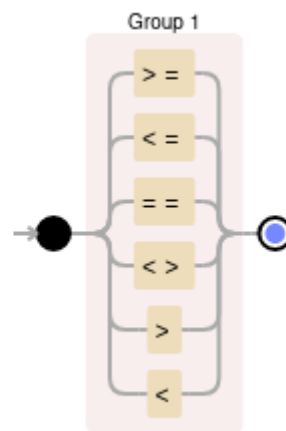
1. Expoente float: Esse token reconhece expoentes de formato E ou e, sendo negativos ou positivos e com qualquer dígito de 0 a 9 uma ou mais vezes (abrangendo todos os números).
2. Caracteres únicos: Esses tokens abrangem os operadores básicos da linguagem e os de sintaxe. Eles incluem: operadores aritméticos, operadores de comparação, a vírgula e a dupla de parênteses.
3. Palavras chaves: Aqui estão a definição dos componentes básicos de uma linguagem, como as estruturas condicionais, as de repetição e os operadores lógicos. Aqui surge a implementação do FOR e do AND e OR no arquivo léxico.
4. Funções pré-definidas: Além do componente básico da linguagem através das palavras chaves, há funções pré-definidas. Aqui está a definição dos tokens para `sqrt`, `exp`, `log` e `print`.
5. Declaração de `NAME` e `NUMBER`: Esses tokens servem para reconhecer, respectivamente, os nomes das funções definidas pelo usuário e os números fornecidos pelo mesmo.
6. Lexemas ignorados: Nessa parte do arquivo, o lexer ignora os comentários do código, novas tabulações e linhas sem fim, e ao ver uma quebra de linha retorna o token EOL e trata o erro de caractere não reconhecido.

Para o diagrama de transição, os tokens escolhidos foram os mais complexos e essenciais para a linguagem:

- a. Token de reconhecimento de comparação
- b. Token de reconhecimento de números
- c. Token de reconhecimento de nome de funções

Segue o diagrama de transição para cada token e uma explicação:

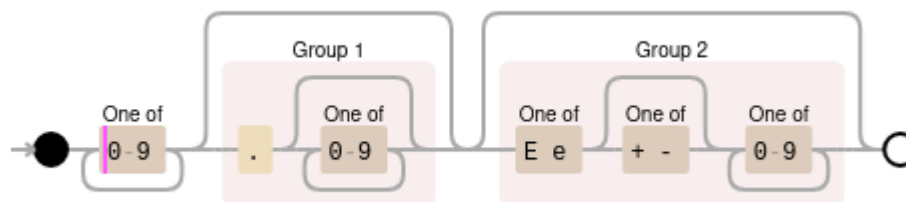
1. Token para comparação (CMP)



Fonte: Autoria própria, desenvolvido no Debuggex

O lexer, ao identificar um operador de comparação, retorna o token de CMP e reconhece a operação de comparação que deve ser executada.

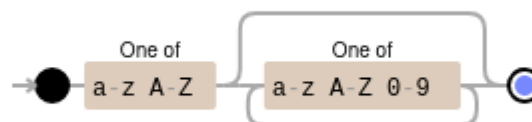
2. Token para números (NUMBER)



Fonte: Autoria própria, desenvolvido no Debuggex

O lexer identifica números inteiros ou fracionários, com expoente float ou não (opcional). Após isso, retorna o token NUMBER.

3. Token para identificadores (NAME)



Fonte: Autoria própria, desenvolvido no Debuggex

Os nomes de funções devem obrigatoriamente começar com uma letra (maiúscula ou minúscula) e seguida de letras ou dígitos. Logo, retorna o token NAME.



Para os outros tokens, como caracteres únicos, palavras reservadas e funções pré definidas, segue uma tabela de descrição e padrões desses tokens. Não foi feito um diagrama para tais pois seria trivial, já que em sua maioria reflete a cópia de seu conteúdo.

Tabela 1. Tokens, padrões e descrições

Token	Padrão	Descrição
IF	if	Comando condicional
THEN	then	Parte do comando IF
ELSE	else	Parte opcional do IF
WHILE	while	Comando de repetição
DO	do	Parte do comando WHILE
FOR	for	Comando de repetição
LET	let	Definição de função
AND	and	Operador lógico E
OR	or	Operador lógico OU
FUNC	sqrt exp log print	Funções pré-definidas
+	+	Adição
-	-	Subtração
*	*	Multiplicação
/	/	Divisão
=	=	Atribuição
,	,	Separador de argumentos
;	;	Separador de comandos
(	(	Abre parêntese
)	)	Fecha parêntese
CMP	> < <= >= == <>	Operadores de comparação
EOL	\n	Fim de linha



–	[\t]	Espaço e tab, ignorados
–	//.*	Comentário, ignorado
–	\\n	Continuação de linha, ignorada

Fonte: Adaptado Claude AI, 2026

2.2. IMPLEMENTAÇÃO NO FLEX

A implementação no flex está presente no arquivo `comp-t1.1` inserido no `.zip` desse trabalho. Nele, todos os tokens são definidos e retornados para o arquivo de parser. Para este relatório, a descrição dos novos tokens exigidos pelo trabalho vão ter prioridade.

Tanto os operadores AND e OR quanto o comando de repetição FOR foram declarados na parte de `palavras` `chaves` no arquivo `lexer`, como segue o trecho no código:

```
"for" {return FOR;} //Adição do for
"and" {return AND;} /* Adição das operações AND e OR */
"or" {return OR;}
```

3. TABELA DE SÍMBOLOS

A tabela de símbolos é a estrutura responsável por armazenar os identificadores usados no programa, como nomes de variáveis e funções definidas pelo usuário. No código, ela é implementada como um vetor fixo chamado `symtab`, com tamanho definido por `NHASH`, e cada posição armazena uma estrutura `symbol`, contendo o nome do identificador, seu valor numérico atual, uma possível árvore de comandos associada a uma função e a lista de parâmetros dessa função. O acesso à tabela é feito pela função `lookup`, que recebe o nome de um símbolo, calcula sua posição inicial por meio de uma função de hash e procura esse nome na tabela. Caso o símbolo já exista, a função retorna sua referência; caso contrário, cria uma nova entrada, inicializando seu valor com zero e seus campos de função como nulos.

Dessa forma, a tabela de símbolos permite que o analisador e o interpretador compartilhem informações sobre variáveis e funções ao longo da execução do programa, mantendo seus valores e definições disponíveis para consultas, atribuições e chamadas futuras.



4. ANÁLISE SINTÁTICA

A análise sintática presente no trabalho baseia-se novamente no código fornecido da calculadora avançada. Ela referencia o código complementar `comp-t1.c`, e isso inclui a utilização da estrutura de dados AST, fundamental para o gerenciamento de comandos e execução dos mesmos. Entretanto, há adaptações em sua lógica ao adicionar os itens de requisitos apresentados na seção 1 do relatório.

4.1. APRESENTAÇÃO DAS REGRAS

O arquivo de parser é dividido em declarações de tokens – gerados pelo lexer, nessa parte os novos tokens de FOR, AND e OR são referenciados. Após isso, é feita a precedência de operações, o que envolve definir a prioridade dos operadores em uma expressão. Como requisitado na descrição do trabalho, os operadores lógicos ocupam a última posição de prioridade, sendo associados à esquerda.

Logo, temos a definição da regra inicial da gramática: `calclist`. Essa regra pode retornar o vazio, a própria regra seguida de comandos da linguagem, a própria regra e a criação de uma função pelo usuário ou a própria regra e um erro (interpretado pelo parser).

A próxima regra `stmt` refere-se aos comandos da linguagem. Aqui, estão definidas as estruturas das condicionais, repetições (incluindo a nova estrutura FOR) ou apenas uma expressão numérica. Para estruturas condicionais e de repetição, temos a referência da regra `list`.

A regra `list` consiste na lista de comandos de execução para determinada estrutura referenciada anteriormente. Para cada comando, um novo nó na árvore AST é criado, com seus respectivos tipos dependendo do comando referenciado. Ao final, ela pode referenciar o vazio, denotando o final da lista dos comandos.

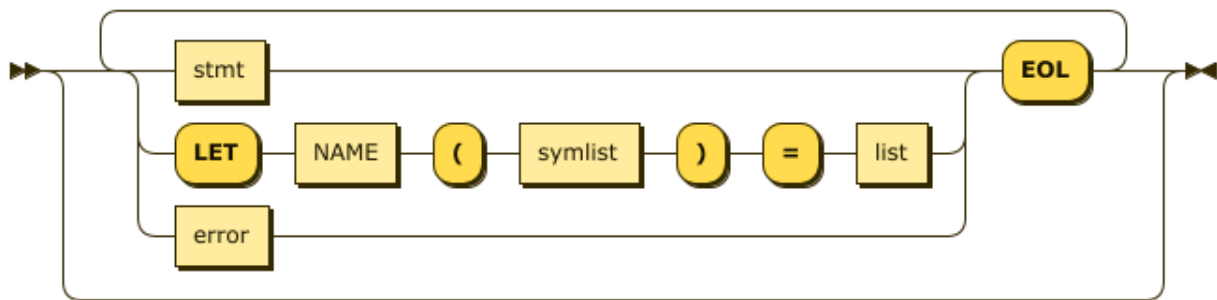
Uma das regras mais importantes é a `exp`, pois ela define todas as operações possíveis de um usuário. Pode ser feita operações comparativas, operações aritméticas, operações lógicas, expressões numéricas, atribuição de variáveis, criação de funções e uso de funções pré-definidas pela linguagem. Além disso, é possível utilizar funções com mais de um argumento, e isso acaba referenciando a regra `explist` para lidar com múltiplos argumentos. Como dito anteriormente, cada regra criada possui o seu retorno em ação no código `.c`, então a AST é alterada com base nos comandos referenciados pela entrada.

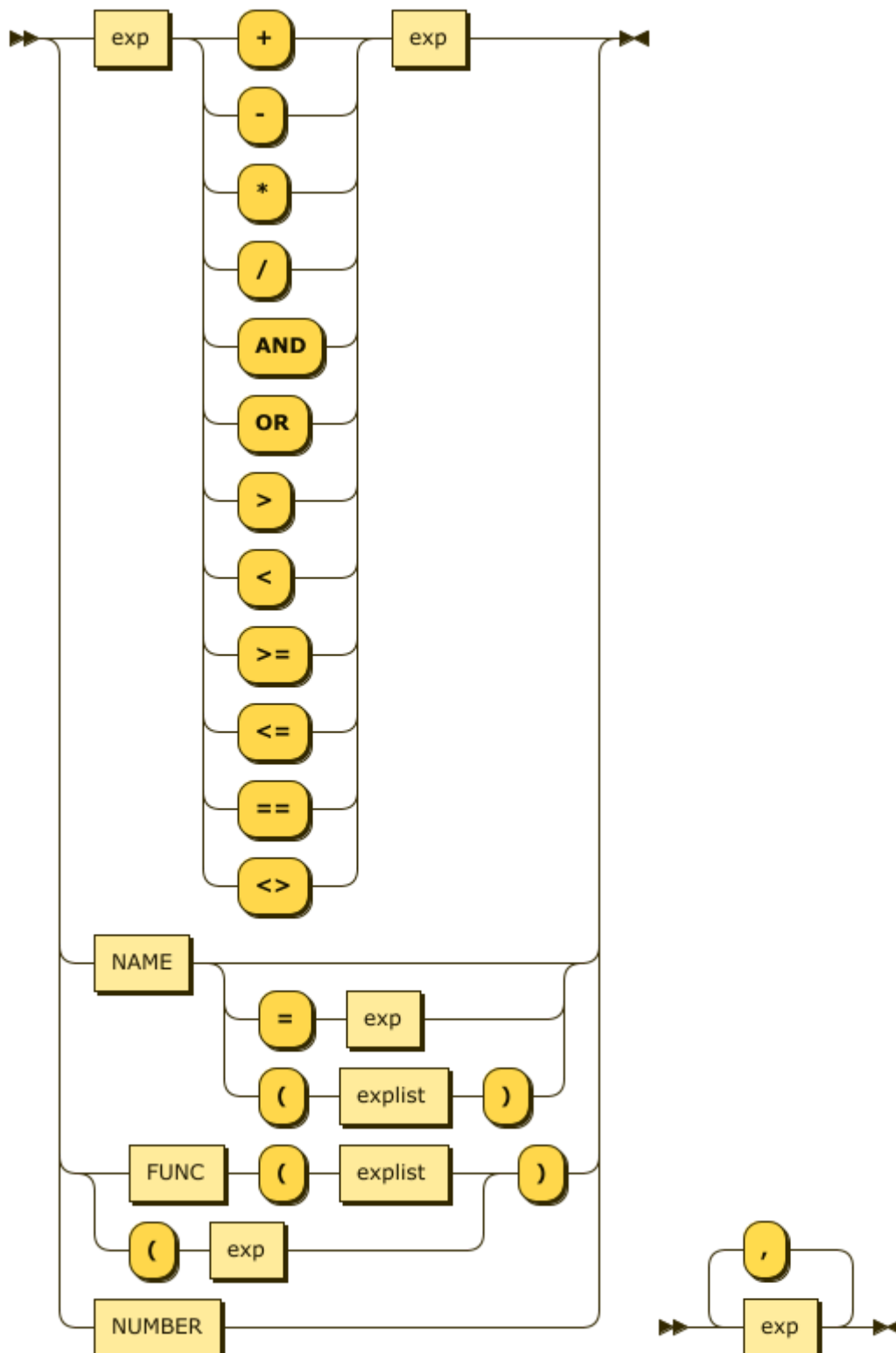


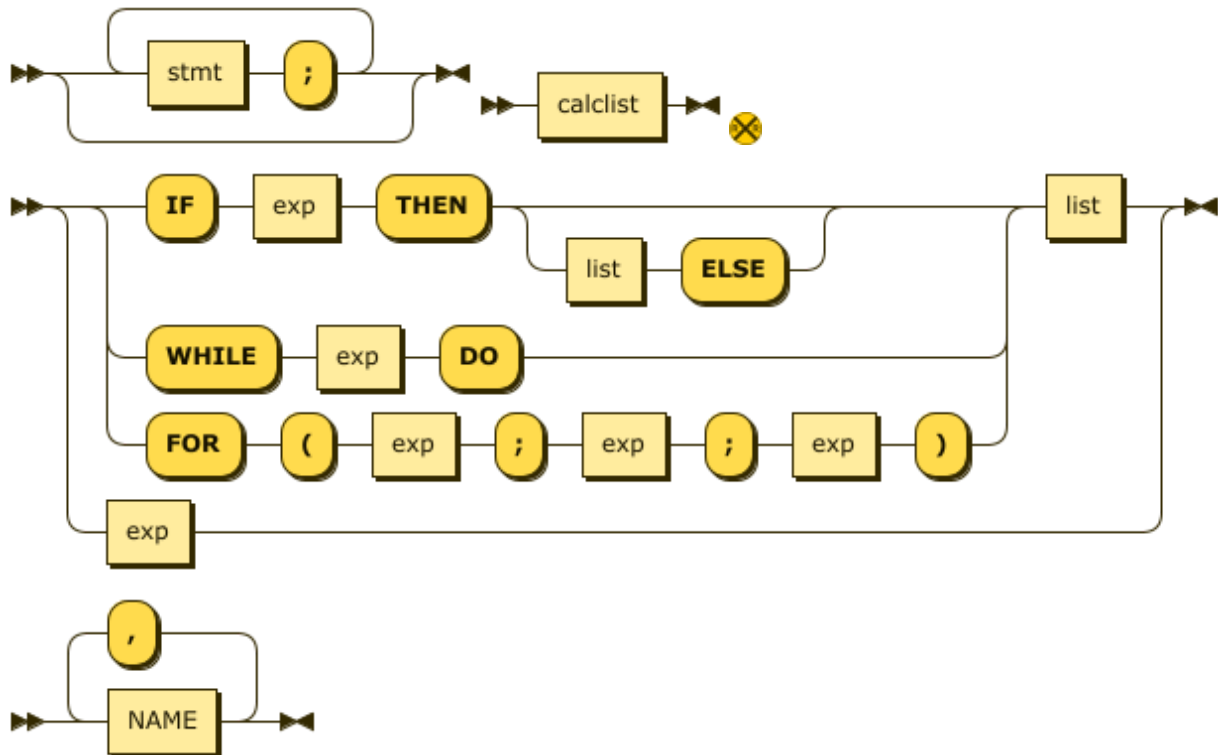
A regra `explist` gerencia os múltiplos argumentos de uma função, ele referencia a si próprio ou a regra `exp` quando em sua análise resta apenas um argumento. O seu funcionamento é semelhante à `list`.

Por último, a regra `symlist` gerencia as variáveis e funções criadas pelo usuário. Ela faz parte da tabela de símbolos, onde esses símbolos são gerenciados e executados através da AST, possuindo então seus próprios tipos em nós.

Para melhor entendimento e visualização sobre as regras em relação à gramática livre de contexto, foi elaborado um diagrama de transição, que especifica todas as regras presentes no arquivo de parser. Esse diagrama foi gerado pela ferramenta **Railroad Diagram** e foi dado de entrada a gramática com a notação EBNF, adaptada da gramática de notação BNF presente na pasta `/apoio-relatorio` no conjunto de arquivos desse trabalho.







Fonte: Autoria própria, desenvolvido no Railroad Diagram

Para melhor visualização dos diagramas, abra o arquivo `diagrama-sintaxe.xhtml` presente na pasta `/apoio-relatorio` do `.zip` desse trabalho.

4.2. IMPLEMENTAÇÃO NO BISON

De forma similar à seção 2.2 que detalha a implementação no Flex, essa seção também prioriza a explicação da adaptação do código para adicionar os itens AND, OR e FOR.

Primeiro, a regra `stmt` foi modificada para comportar a regra do FOR. Nela, foi especificada a sintaxe da função pré-definida e o seu comportamento referenciado no código C. A regra pode ser consultada abaixo, e a modificação está sinalizada no próprio código.

```
stmt: IF exp THEN list { $$ = newflow('I', $2, $4, NULL); }  
    | IF exp THEN list ELSE list { $$ = newflow('I', $2, $4, $6); }  
    | WHILE exp DO list { $$ = newflow('W', $2, $4, NULL); }  
    | FOR '(' exp ';' exp ';' exp ')' stmt { $$ = newfor('R', $3, $5,  
$7, $9); } /* NOVA REGRA: COMANDO FOR */  
    | exp
```



;

Podemos ver que a regra respeita a sintaxe definida pela descrição do trabalho e as duas regras `exp` presentes na estrutura refere-se, respectivamente, às variáveis `init`, `cond` e `inc`. Dessarte, é indicado um novo comportamento a ser implementado no código .c, a função `newfor()`, que atende ao gerenciamento da AST e executa sua rotina.

Além disso, a implementação dos operadores lógicos `AND` e `OR`, inicia-se pela definição de sua precedência e associação, como pode ser visto nesse trecho de código, adaptado para esses itens.

```
%right '='  
%left OR  
%left AND  
%nonassoc <fn> CMP  
%left '+' '-'  
%left '*' '/'
```

Após, o código segue com a definição da operação lógica a partir da regra `exp`, onde declara o reconhecimento de sua operação e o seu comportamento. A regra `exp` adaptada pode ser consultada abaixo.

```
exp: exp CMP exp { $$ = newcmp($2, $1, $3); }  
    | exp '+' exp { $$ = newast('+', $1, $3); }  
    | exp '-' exp { $$ = newast('-', $1, $3); }  
    | exp '*' exp { $$ = newast('*', $1, $3); }  
    | exp '/' exp { $$ = newast('/', $1, $3); }  
    | exp AND exp { $$ = newast('A', $1, $3); } /* ADIÇÃO AND */  
    | exp OR exp { $$ = newast('O', $1, $3); } /* ADIÇÃO OR */  
    | '(' exp ')' { $$ = $2; }  
    | NUMBER { $$ = newnum($1); }  
    | NAME { $$ = newref($1); }  
    | NAME '=' exp { $$ = newasgn($1, $3); }  
    | FUNC '(' explist ')' { $$ = newfunc($1, $3); }  
    | NAME '(' explist ')' { $$ = newcall($1, $3); }  
;  
;
```

Similar ao novo comando `FOR`, a operação lógica adiciona dois novos tipos de nó da AST e seus comportamentos.



5. CONJUNTO DE TESTES

Para testar a linguagem de programação básica, foram feitos alguns testes iniciais simples como print de string, declaração e atribuição de variáveis. Como o código apresentado na calculadora avançada não abrangia a funcionalidade de imprimir strings na tela, foi necessária uma adaptação nos arquivos Flex, Bison e em C para ser possível imprimir essas mensagens na saída do programa.

Após isso, foi feita outra adaptação nos arquivos para o código em C aceitar arquivos em texto como entrada para o programa. Dessa forma, ao ler um arquivo `.txt`, o programa em C pega esse conteúdo, analisa lexicalmente e sintaticamente e executa as operações requeridas.

O arquivo `programa.txt`, o qual se refere ao conjunto de comandos de teste para esse trabalho, está disponível juntamente com os arquivos de código fonte. O projeto pode ser iniciado através dos comandos:

```
make  
./comp-t1
```

Também é possível criar outro arquivo de teste como `.txt` e ao executar o `./comp-t1`, informar o caminho do arquivo `.txt`. Como consta abaixo:

```
./comp-t1 caminho/do/programa.txt
```

Por padrão, ao executar o `./comp-t1`, o arquivo de teste dado como entrada é o `programa.txt`.

O resultado do teste está abaixo:



```
fernanda@fernanda-Vivobook-ASUSLaptop-M1502YA-M1502YA:~/Documentos/GitHub/COMP-T1-FernandaPachecoBento$ ./comp-t1
Início do programa
= 7
= 10
= 0
Valores iniciais:
= 7
= 10
= 0
Teste do operador AND:
= 1
Teste do operador OR:
= 1
Teste combinando atribuicao, comparacao e logica:
= 1
= 1
Teste do comando FOR:
= 0
= 1
= 2
= 3
= 4
= 4
Teste do FOR com acumulador:
= 0
= 15
= 15
Teste do IF com AND:
soma maior que 10 e x menor que y
= 0
Teste do WHILE com OR:
= 0
= 4
= 4
Fim do programa
```

6. CONSIDERAÇÕES FINAIS

Dessarte, torna-se válido reconhecer que o presente trabalho atende aos requisitos da descrição. Uma vez que foi possível implementar o novo comando FOR da linguagem com a sintaxe definida, e as operações lógicas a partir do AND e OR. Os testes foram concluídos com sucesso e apresentaram resultados satisfatórios.

7. USO DE IA

Para realizar esse trabalho, a Inteligência Artificial foi utilizada para acelerar o processo de codificação e correção de erros no código. Abaixo, segue um detalhamento de quais ferramentas de LLM foram utilizadas, como foram utilizadas e para o que foram utilizadas.

Claude (Anthropic):

- Auxílio: definição da linguagem de programação básica, criação das operações AND e OR na regra exp, criação da tabela de tokens e padrão, cópia do código .h da calculadora avançada.



-
- Organização do plano de ação para o trabalho.
 - Correção: tokens AND e OR, definição da nova regra FOR, gramática em BNF, diagrama de sintaxe, arquivo lexer.

Codex (OpenAI):

- Auxílio: cópia do código .c da calculadora avançada, adaptação para o arquivo de entrada e impressão de strings, criação do arquivo teste de entrada.
- Correção: implementação do novo comando FOR e precedência dos operadores AND e OR.